



BITS Pilani

ML SYSTEM OPTIMIZATION





M1: Foundations



Performance Metrics & Complexity Analysis



1. *Time/space complexity*
2. *Throughput*
3. *Latency*
4. *Speedup (Amdahl's Law)*

Time Complexity

- Time complexity measures how the **execution time of an algorithm** grows with the size of input n . It is independent of hardware but depends on the algorithm's structure.

$T(n)$ = time taken to process input of size n

- Helps **predict performance** for large inputs.
- Allows comparison between **different algorithms** solving the same problem.
- Guides **optimization** in high-performance computing.

Asymptotic Notations

Big O (O): Upper bound, worst-case time

$T(n) = O(f(n)) \Rightarrow$ Algorithm will take at most $c \cdot f(n)$ time for large n

Omega (Ω): Lower bound, best-case time

$T(n) = \Omega(f(n)) \Rightarrow$ Algorithm will take at least $c \cdot f(n)$ time for large n

Theta (Θ): Tight bound, average-case time

$T(n) = \Theta(f(n)) \Rightarrow$ Algorithm grows exactly like $f(n)$

Time Complexity

Algorithm	Best Case	Average Case	Worst Case
Linear Search	$\Omega(1)$	$\Theta(n)$	$O(n)$
Binary Search	$\Omega(1)$	$\Theta(\log n)$	$O(\log n)$
Merge Sort	$\Omega(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$
Bubble Sort	$\Omega(n)$	$\Theta(n^2)$	$O(n^2)$

Common Growth Rates

- **$O(1)$** : Constant time – independent of n
- **$O(\log n)$** : Logarithmic – e.g., binary search
- **$O(n)$** : Linear – e.g., linear search, sum of array
- **$O(n \log n)$** : Log-linear – e.g., merge sort, quicksort average case
- **$O(n^2)$** : Quadratic – e.g., bubble sort, insertion sort
- **$O(2^n)$, $O(n!)$** : Exponential / factorial – impractical for large n

Time Complexity

Linear Search: Search for element in array of size n

$$T(n) = \text{comparisons made} = n \text{ (worst case)}$$

$$T(n) = O(n)$$

Binary Search: Search in sorted array

$$T(n) = 1 + 1 + \dots + \log_2 n = O(\log n)$$

Recursive Algorithms

Time complexity can also be **recursively defined**. Example: Merge Sort

$$T(n) = 2 \cdot T\left(\frac{n}{2}\right) + O(n)$$

$$T(n) = 2 \cdot T(2n) + O(n)$$

Using Master Theorem: $T(n) = O(n \log n)$

Space Complexity

Space complexity measures **how much memory an algorithm requires** relative to input size n . It includes:

1. **Input space:** Memory needed to store inputs
2. **Auxiliary space:** Temporary space used by algorithm
3. **Function/recursion stack:** Memory for recursion calls

$$S(n) = S_{input} + S_{auxiliary} + S_{stack}$$

Why Space Complexity Matters

- Helps predict **memory requirements** for large datasets.
- Prevents **memory overflows** in HPC and parallel systems.
- Guides algorithm selection for memory-constrained systems.

Space Complexity

Linear Search: Only a few variables used for index and element comparison.

$$S(n)=O(1)(\text{constant space})$$

Merge Sort: Uses an auxiliary array for merging.

$$S(n)=O(n)(\text{extra array of size } n)$$

Quick Sort (in-place): Recursion stack used: $\log n$ depth on average

$$S(n)=O(\log n)(\text{average case})$$

Matrix Multiplication: For two $n \times n$ matrices:

$$S(n)=O(n^2)(\text{stores output matrix})$$

Space Complexity

If an algorithm uses:

- k scalar variables $\rightarrow$ each takes constant space $\rightarrow$ total $O(k)$
- n input elements $\rightarrow$ space grows with input size $\rightarrow O(n)$
- Recursion depth $d \rightarrow$ each call uses stack space $\rightarrow$ total $O(d)$

Then:

$$S(n) = O(n + k + d)$$

All these memory usages happen together, so we **add them**. In Big-O, we keep the dominant growing term. If **k is constant** and **$d \leq n$** , then:

$$S(n) = O(n)$$

because **n grows fastest and dominates the total space**.

Throughput

Throughput is the **number of tasks or operations completed per unit time**. It measures **system-wide productivity**.

$$\text{Throughput} = \frac{\text{Number of Tasks Completed}}{\text{Total Time}}$$

Example (Sequential System):

- CPU processes 1 task every 0.25 sec
- For 100 tasks:

$$\text{Throughput} = \frac{100}{100 \times 0.25} = 4 \text{ tasks/sec}$$

In Parallel Systems:

- Throughput can increase with **more cores/processors**
- Ideal case (linear scaling):

$$\text{Throughput}_{\text{parallel}} = N \times \text{Throughput}_{\text{single core}}$$

where N = number of cores

Latency

Latency is the **time taken to complete a single task from start to finish.**

$$\text{Latency} = \text{Finish Time} - \text{Start Time}$$

Example:

- Task A starts at 0 sec, finishes at 2 sec → Latency = 2 sec
- Task B starts at 1 sec, finishes at 3 sec → Latency = 2 sec

Important Note:

- Low latency $\neq$ high throughput (especially in parallel systems)
- Example: Batch processing can increase throughput but individual task latency may remain high

Relationship Between Throughput and Latency

Little's Law (Queueing Theory)

$$L = \lambda \cdot W$$

Where:

- L = Average number of tasks in the system (system load)
- λ = Arrival rate (tasks/sec) → closely related to **throughput**
- W = Average time spent in the system → **latency**

Rearranging:

$$W = \frac{L}{\lambda}$$

- If arrival rate λ increases
- But system resources (CPU, memory, cores) remain fixed

As tasks begin to queue, the load (L) increases, which in turn increases the latency (W).

Relationship Between Throughput and Latency

Increasing throughput does **not automatically reduce latency**.

If system utilization approaches 100%:

- Queue length grows rapidly
- Waiting time increases
- Latency increases nonlinearly

In many queue models (e.g., M/M/1): $W = \frac{1}{\mu - \lambda}$

Where:

- μ = service rate
- As $\lambda \rightarrow \mu$

$$W \rightarrow \infty$$

So latency grows dramatically near saturation.

Core Performance Metrics

$$\text{Throughput} = \frac{\text{Tasks Completed}}{\text{Total Time}} \quad (\text{tasks/sec})$$

$$\text{Latency} = \text{Task Completion Time} - \text{Task Arrival Time} \quad (\text{seconds})$$

$$\text{Efficiency} = \frac{\text{Throughput}_{\text{actual}}}{\text{Throughput}_{\text{ideal}}}$$

In parallel systems:

$$\text{Efficiency} = \frac{\text{Speedup}}{p}$$

Where:

$$\text{Speedup} = \frac{T_1}{T_p}$$

- T_1 = execution time on 1 processor
- T_p = execution time on p processors
- p = number of processors

Speedup

Speedup measures how much faster a parallel system performs compared to serial execution.

$$S = \frac{T_{serial}}{T_{parallel}}$$

Where:

- T_{serial} = execution time on **one processor**
- $T_{parallel}$ = execution time on **N processors**

Example

- Single-core time: $T_{serial} = 100$ sec
- 4-core time: $T_{parallel} = 30$ sec

Speedup

- The system is **3.33 times faster** using 4 processors.
- Ideal speedup for 4 processors would be:

$$S_{ideal} = 4$$

Since $3.33 < 4$, the system has some overhead (communication, synchronization, imbalance).

Important Insight

$$S \leq N$$

- If $S = N \rightarrow$ Perfect linear speedup
- If $S < N \rightarrow$ Parallel overhead exists
- If $S > N \rightarrow$ Superlinear speedup (rare, due to cache effects)

Amdahl's Law

Amdahl's Law predicts the **maximum theoretical speedup** of a program when only part of it can be parallelized.

Assumptions

- P = fraction of program that can run in parallel
- $1-P$ = fraction that must run sequentially
- N = number of processors

Parallel Execution Time

Total serial time = T_{serial}

Sequential portion time:

$$(1-P)T_{\text{serial}}$$

Parallel portion time (divided among N processors):

$$\frac{PT_{\text{serial}}}{N}$$

So total parallel time:

$$T_{\text{parallel}} = T_{\text{serial}} \left[(1-P) + \frac{P}{N} \right]$$

Speedup

$$S(N) = \frac{T_{\text{serial}}}{T_{\text{parallel}}} = \frac{1}{(1-P) + \frac{P}{N}}$$

Amdahl's Law

1. The **sequential part (1 - P)** must always execute on a single processor.
It cannot be reduced by adding more processors.
2. The **parallel part (P)** gets divided across N processors, so its time reduces as N increases.
3. However, even if $N \rightarrow \infty$:

$$S_{max} = \frac{1}{1 - P}$$

This means the **sequential fraction limits the maximum speedup**.

Example:

If:

- $P=0.9$ (90% parallelizable)

Then:

$$S_{max} = 1 / 0.1 = 10$$

Even with unlimited processors, speedup cannot exceed 10.

Amdahl's Law

Given:

- Parallel fraction: $P = 0.8$
- Sequential fraction: $1 - P = 0.2$
- Number of processors: $N = 4$

Apply Amdahl's Law

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

Substitute values:

$$S(4) = \frac{1}{0.2 + \frac{0.8}{4}}$$

$$S(4) = \frac{1}{0.2 + 0.2}$$

$$S(4) = \frac{1}{0.4}$$

$$S(4) = 2.5$$

Even though 4 processors are used, the speedup is only **2.5×**, not 4×

Why?

- The 20% sequential portion must still run on one processor.
- Only the 80% portion benefits from parallel execution.
- This sequential part becomes a limiting factor.

Amdahl's Law

If total serial time = 100 sec:

- Sequential part (20%) = 20 sec
- Parallel part (80%) = 80 sec

With 4 processors:

Parallel time: $80 / 4 = 20$ sec

Total parallel execution time: $20 + 20 = 40$ sec

Speedup: $100 / 40 = 2.5$

The sequential portion $(1-P)$ restricts scalability. Adding more processors improves performance, but the improvement is limited by the non-parallelizable part of the program.

Amdahl's Law

Suppose a video rendering task takes **200 minutes** on a single processor.

- 70% of the work (frame rendering) can be parallelized

$$P = 0.7$$

- 30% (final encoding + synchronization) is sequential

$$1 - P = 0.3$$

- Number of processors:

$$N = 8$$

Apply Amdahl's Law

$$S(8) = \frac{1}{(1 - P) + \frac{P}{N}}$$

$$S(8) = \frac{1}{0.3 + \frac{0.7}{8}}$$

$$S(8) = \frac{1}{0.3 + 0.0875}$$

$$S(8) = \frac{1}{0.3875}$$

$$S(8) \approx 2.58$$

Amdahl's Law

Original time = 200 minutes

Speedup ≈ 2.58

New execution time:

$$T_{parallel} = \frac{200}{2.58} \approx 77.5 \text{ minutes}$$

Why Not 8× Speedup?

Even though 8 processors are used:

- The 70% rendering part is divided across 8 cores.
- But the 30% encoding part must still run sequentially.

Amdahl's Law

Sequential time:

$$0.3 \times 200 = 60 \text{ minutes}$$

That 60 minutes cannot be reduced by adding more processors.

In real systems like video rendering:

- Parallel tasks (frame processing) scale well.
- Sequential tasks (final merge, encoding) limit total speedup.

So even with powerful HPC clusters, performance is bounded by the sequential bottleneck.

This is exactly what Amdahl's Law predicts.

Maximum Speedup

We know:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

To find the **maximum possible speedup**, take the limit as the number of processors becomes very large:

$$\lim_{N \rightarrow \infty} S(N)$$

Apply the Limit

As:

$$N \rightarrow \infty$$

The term:

$$\frac{P}{N} \rightarrow 0$$

So the speedup formula becomes:

$$S_{max} = \frac{1}{(1 - P)}$$

Maximum Speedup

Suppose total serial execution time = 100 seconds.

- Parallel part (80%) = 80 sec
- Sequential part (20%) = 20 sec

If we use infinite processors:

- Parallel part becomes ≈ 0 sec
- Sequential part remains 20 sec

So total time ≈ 20 sec.

Speedup: $100 / 20 = 5$

Even with unlimited processors, the execution time **cannot go below 20 seconds**, because that portion cannot be parallelized.



Efficiency

Efficiency measures how well the processors are utilized during parallel execution.

$$E(N) = \frac{S(N)}{N}$$

Where:

- $S(N)$ = speedup with N processors
- N = number of processors

It tells us what fraction of each processor's capacity is effectively used.

Using Amdahl's Law

We know:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

Substitute into efficiency:

$$E(N) = \frac{1}{N \left[(1 - P) + \frac{P}{N} \right]}$$

Simplify:

$$E(N) = \frac{1}{N(1 - P) + P}$$

Efficiency

Given:

- $P = 0.8$
- $N = 4$

Previously calculated speedup:

$$S(4) = 2.5$$

Efficiency:

- $E(4) = 2.54 / 4$
- $E(4) = 0.625$
- $E(4) = 62.5\%$

Although 4 processors are used:

- The system achieves only $2.5\times$ speedup.
- So each processor is effectively utilized at 62.5%.

Although 4 processors are used:

- The system achieves only $2.5\times$ speedup.
- So each processor is effectively utilized at 62.5%.
- The remaining 37.5% is wasted due to the sequential part $(1-P)$ of the code.

Why Efficiency < 1

- Sequential fraction of the program $(1 - P = 0.2)$ cannot be parallelized.
- Adding more processors **does not speed up sequential tasks**, so efficiency drops.
- With more processors, efficiency **keeps decreasing**, illustrating **diminishing returns**.

Efficiency

Why Efficiency Decreases as N Increases

From:

$$E(N) = \frac{1}{N(1 - P) + P}$$

As N increases:

- The term $N(1-P)$ increases.
- Denominator increases.
- So efficiency decreases.



Unless:

$P \rightarrow 1$

Meaning the program is almost fully parallelizable



Parallelization Paradigms



1. *Data-level parallelism*
2. *Task-level parallelism*
3. *Algorithm-specific parallelism*

Parallelization

Parallelization is the decomposition of a computational workload into independent components that can execute concurrently on N processors.

Let T_{serial} be the total execution time of a task on a single processor.

If a fraction P of the task is parallelizable and is distributed across N processors, then the parallel execution time is:

$$T_{\text{parallel}} = (1 - P)T_{\text{serial}} + \frac{PT_{\text{serial}}}{N}$$

Ideal Case (No Sequential Part, No Overhead)

If the entire task is parallelizable ($P=1$): $T_{\text{parallel}} = \frac{T_{\text{serial}}}{N}$

Speedup: $S(N) = \frac{T_{\text{serial}}}{T_{\text{parallel}}} = N$

*This is called **linear speedup***

Parallelization

General Case (With Overhead)

Including communication and synchronization overhead T_{overhead} :

$$T_{\text{parallel}} = (1 - P)T_{\text{serial}} + \frac{PT_{\text{serial}}}{N} + T_{\text{overhead}}$$

Objective of Parallelization

Minimize:

$$T_{\text{parallel}}$$

Maximize:

$$S(N) = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

and maintain high efficiency:

$$E(N) = \frac{S(N)}{N}$$

Parallelization

Parallelization attempts to transform a computation of cost:

$$O(T)$$

into:

$$O(T / N)$$



subject to limits imposed by sequential fractions and system overhead

Work Integrated Learning Programmes

Data-Level Parallelism (DLP)

Data-Level Parallelism refers to performing the **same operation** on multiple data elements simultaneously.

It is applicable when:

- The operation on each data element is **independent**
- No inter-element dependency exists
- Data can be partitioned evenly

Given two arrays A and B of size n:

$$C[i] = A[i] + B[i], i = 1, 2, \dots, n$$

Each computation of $C[i]$ is independent of others. So the total serial execution time:

$$T_{\text{serial}} = n \cdot t_{\text{op}}$$

Where:

- t_{op} = time for one addition

Data-Level Parallelism (DLP)

Parallel Execution

If N processors are available and work is evenly distributed:

$$T_{parallel} = \frac{n}{N} \cdot t_{op}$$

Speedup:

$$S(N) = \frac{T_{serial}}{T_{parallel}} = \frac{nt_{op}}{\frac{n}{N}t_{op}} = N$$

Data-Level Parallelism (DLP)

Maximum Speedup

If $N = n$ (one processor per element):

$$T_{parallel} = t_{op}$$

$$S_{max} = n$$

In real systems:

- There is communication overhead
- Memory access delays occur
- Processors may not be perfectly synchronized

So practical speedup is slightly less than n .

Thus we write:

$$S_{max} \approx n$$

meaning **approximately linear speedup**.

Data-Level Parallelism (DLP)

Computational Complexity View

Serial Complexity

$$\text{Serial Time Complexity} = O(n)$$

Parallel Complexity (Ideal Case, No Overhead)

$$\text{Parallel Time Complexity} = O(n / N)$$

Where:

- n = problem size
- N = number of processors

If $N = n$

$$O(n / n) = 1$$

This represents **constant time complexity** in the ideal case (perfect parallelism, no communication overhead).

Data-Level Parallelism (DLP)

Image Processing

Applying a filter (e.g., blur, edge detection) to each pixel can be done independently. Each processor/thread handles a subset of pixels simultaneously.

Vector Addition / Matrix Operations

Operations such as:

$$C[i] = A[i] + B[i]$$

can be computed independently for each index i . Similarly, matrix multiplication distributes row-column computations across multiple processors.

GPU Computations

Modern GPUs such as those using **CUDA architecture by NVIDIA** allow thousands of cores to execute threads in parallel.

For example:

- Each CUDA core handles one data element.
- Large-scale computations (deep learning, simulations, rendering) achieve massive speedup.

Task-Level Parallelism (TLP)

Task-Level Parallelism (TLP) refers to executing **multiple independent tasks simultaneously** instead of executing them sequentially.

- Each task performs a different operation.
- Tasks may work on the same dataset or different datasets.
- The goal is to reduce overall completion time by overlapping execution.

Unlike data parallelism (where the same operation is applied to different data elements), TLP focuses on parallelizing **different functions or modules**.

Task-Level Parallelism (TLP)

In **TLP**, we divide the program into **independent tasks**:

$$T_1, T_2, \dots, T_m$$

Each task performs a logically different unit of work.

Unlike data parallelism:

- Tasks may operate on **different data**
- Tasks may even execute **different algorithms**

Sequential Execution Time

If executed sequentially:

$$T_{\text{serial}} = \sum_{i=1}^m t_i$$

- $t_i \rightarrow$ execution time of task T_i
- Summation means tasks execute **one after another**
- Total time is additive

Task-Level Parallelism (TLP)

Assume we have m tasks:

$$T_1, T_2, \dots, T_m$$

Each task takes time:

$$t_1, t_2, \dots, t_m$$

Since serial execution runs tasks one by one:

$$T_{\text{serial}} = t_1 + t_2 + \dots + t_m$$

Then,

$$T_{\text{serial}} = \sum_{i=1}^m t_i$$

Because:

- Task 2 starts only after Task 1 finishes.
- Task 3 starts only after Task 2 finishes.
- No overlap happens.

*So total time is the **sum of all task durations**.*

Task-Level Parallelism (TLP)

Parallel Execution Time

Now assume we have **m processors**, and each task runs on one processor.

All tasks start at the same time.

The program finishes when the **last task finishes**.

So:

$$T_{\text{parallel}} \approx \max(t_1, t_2, \dots, t_m)$$

Task	Time
T ₁	5 sec
T ₂	3 sec
T ₃	2 sec

All start together.

- T₃ finishes at 2 sec
- T₂ finishes at 3 sec
- T₁ finishes at 5 sec

System completes at 5 sec. That is the **maximum time**.

*This is called the **critical path time**.*

Speedup in Task-Level Parallelism (TLP)

Speedup measures how much faster the parallel system is compared to serial execution.

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}}$$

- T_{serial} → Total time taken when tasks are executed one after another.
- T_{parallel} → Total time taken when tasks are executed simultaneously.

Why Serial divided by Parallel?: Because we want to measure improvement.

$$\text{Speedup} = \frac{\text{Old Time}}{\text{New Time}}$$

- Old time → Serial time
- New time → Parallel time

If:

$$T_{\text{serial}} = 100 \text{ seconds}$$

$$T_{\text{parallel}} = 20 \text{ seconds}$$

Then:

$$S = 100 / 20 = 5$$

*The parallel system is **5 times faster**.*

Load Balance Ratio in Task-Level Parallelism (TLP)

Average Task Time

$$\bar{t} = \frac{1}{m} \sum_{i=1}^m t_i$$

- $m \rightarrow$ number of tasks
- $t_i \rightarrow$ execution time of task i
- $\bar{t} \rightarrow$ average execution time per task

If total work is divided equally among tasks, each task should ideally take $\bar{t}$ time.

It represents the **ideal balanced workload per processor**.

Load Balance Ratio in Task-Level Parallelism (TLP)

Ideal Parallel Time

$$T_{\text{ideal}} = \bar{t}$$

If:

- All tasks are perfectly balanced
- Each processor gets equal work

Then every task finishes at the same time.

So total completion time = average task time.

*This is the **best possible parallel time**
(assuming no overhead).*

Actual Parallel Time

$$T_{\text{parallel}} = \max(t_i)$$

Even if most tasks are small:

- The slowest task determines when the system finishes.
- Faster processors must wait for the slowest one.

*This is the **bottleneck effect**.*

Task-Level Parallelism (TLP)

When Number of Tasks $\neq$ Number of Processors

m = number of tasks

p = number of processors

*If $m > p \rightarrow$ All tasks cannot execute simultaneously, so tasks must be **scheduled***

Parallel Time with Scheduling

$$T_{\text{parallel}} = \max_{j=1}^p \left(\sum t_i \text{ assigned to processor } j \right)$$

- Each processor gets multiple tasks
- Total time per processor = sum of assigned task times
- Overall completion time = maximum workload among processors

When $m > p$, performance depends more on intelligent scheduling than on the number of processors.

Task-Level Parallelism (TLP)

Adding Overhead

$$T_{parallel} = \max(t_i) + T_{overhead}$$

- $\max(t_i)$ → Longest task (critical path)
- $T_{overhead}$ → Extra system costs

Overhead Includes

- Communication delay
- Synchronization (locks / barriers)
- Task or thread creation
- Context switching

Speedup with Overhead

$$S = \frac{\sum t_i}{\max(t_i) + T_{overhead}}$$

Algorithm-Specific Parallelism (ASP)

Algorithm-Specific Parallelism refers to designing **parallel algorithms tailored to the mathematical structure of a specific problem.**

- Unlike generic data-level or task-level parallelism, ASP **exploits the algorithm's inherent recursion, decomposition, or structure** for concurrency.
- Often used in **scientific computing, numerical methods, and signal processing.**

Instead of simply dividing tasks or data, we analyze:

- Recurrence relations
- Decomposition structure
- Algebraic independence
- Divide-and-conquer properties

*Parallelism is extracted from the **theory of the algorithm itself.***

Parallel Sorting

Serial Merge Sort

Recurrence relation:

$$T_{\text{serial}}(n) = 2 T(n/2) + O(n)$$

- Divide array in two halves
- Sort each half sequentially
- Merge sorted halves in $O(n)$ time

Parallel Merge Sort

If two halves are sorted **simultaneously on 2 processors**:

Where: $T_{\text{parallel}}(n) = \frac{2 T(n/2)}{N} + O(n)$

- N = number of processors used for the recursive step
- Merge step still requires $O(n)$ time

Parallel Sorting

Suppose $N=2$, array size $n=8$:

1. Divide $\rightarrow$ 2 subarrays of size 4, sorted **concurrently**
2. Each subarray further divides $\rightarrow$ recursion continues
3. Merge step at each level takes $O(n)$

Parallelization reduces the **depth of recursion** execution time

Speedup is limited by the **merge step** (sequential part)

This is an example of **Amdahl's law applied recursively**.

Matrix Multiplication

$$C = A \cdot B$$

Divide A and B into submatrices (blocks)

Compute block products independently:

$$C_{ij} = \sum_k A_{ik} B_{kj}$$

Each block multiplication is independent → execute in parallel

Strassen Algorithm (Parallel): Strassen reduces number of multiplications using recursion

$$T_{\text{serial}}(n) = 7T(n/2) + O(n^2)$$

Parallel form:

$$T_{\text{parallel}}(n) = \frac{7T(n/2)}{N} + O(n^2)$$

- N processors handle recursive multiplications simultaneously
- Significant speedup, especially on GPUs

Algorithm-Specific Parallelism (ASP)

Most algorithm-specific parallel algorithms follow:

$$T_{\text{parallel}}(n) = \frac{\text{Recursive Subproblems}}{N} + T_{\text{combine}}$$

Where:

- Recursive subproblems → can be executed concurrently
- T_{combine} → sequential portion for merging or summing results

Speedup:

$$S = \frac{T_{\text{serial}}}{T_{\text{parallel}}} \leq \frac{T_{\text{serial}}}{T_{\text{combine}} + T_{\text{parallel-subtasks}}}$$

- Limited by **sequential combine step** (Amdahl's Law)

Algorithm-Specific Parallelism (ASP)

Algorithm-specific parallelism **maximizes concurrency** by exploiting the problem's structure

Algorithm	Parallel Strategy
Merge Sort	Sort halves concurrently
FFT	Recursive even-odd decomposition
Matrix Multiplication	Block multiplication / Strassen
Numerical PDE Solvers	Domain decomposition

Efficiency depends on:

- Number of processors N
- Sequential combine time
- Task granularity

Parallelization Paradigms

Paradigm	Concept	Example	Hardware Best Suited
Data-Level Parallelism (DLP)	Same operation on multiple data elements	Vector addition, image filter	GPUs, SIMD units
Task-Level Parallelism (TLP)	Different tasks executed concurrently	Web server requests, pipeline	Multi-core CPUs, clusters
Algorithm-Specific	Parallel algorithms for problem structure	Parallel merge sort, FFT	CPU cores, GPUs, clusters



Modern Hardware Architectures



1. *Multi-core CPUs*
2. *GPGPUs*
3. *GPU/CPU Clusters*
4. *Resource Management*

Multi-core CPUs

A **multi-core CPU** integrates multiple independent processing cores on a single chip.

Each Core Contains:

- Private **registers**
- **ALU (Arithmetic Logic Unit)**
- Often private **L1 cache**
- Shared **L2/L3 cache**
- Shared **main memory (RAM)**

Parallel Model → Shared Memory Model

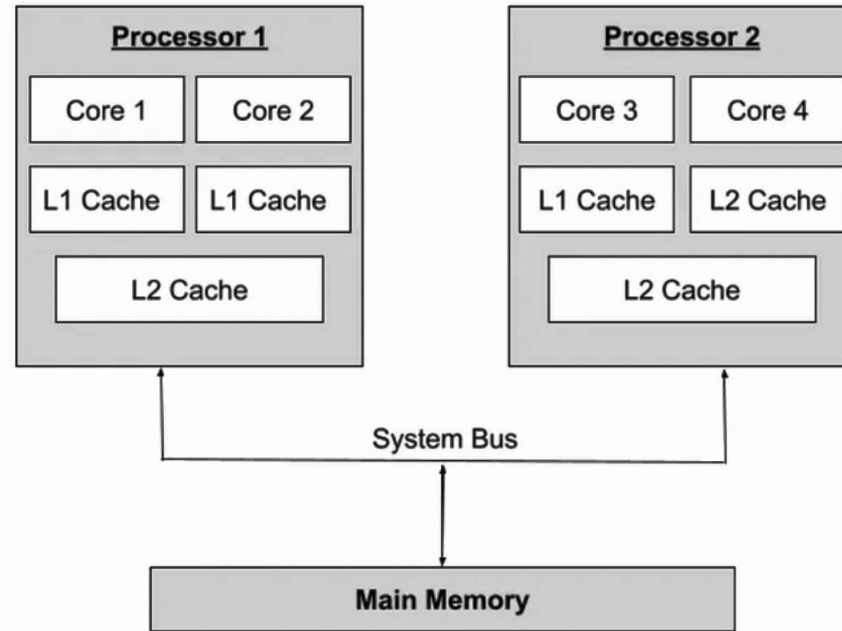
All cores access the same global address space:

$$M_{global} \rightarrow \{Core_1, Core_2, \dots, Core_N\}$$

This enables:

- Thread-based programming (e.g., OpenMP, Pthreads)
- Fast inter-thread communication (via shared variables)

Multi-core CPUs

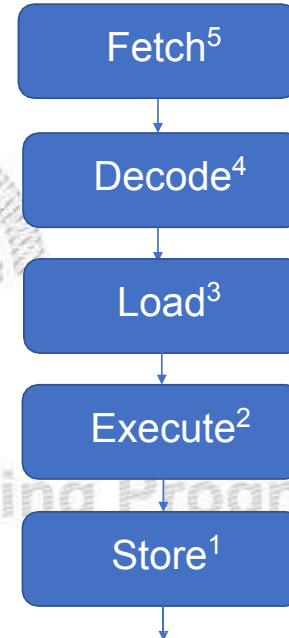


Multi-core CPUs - Instruction Pipeline

Given a sequence of instructions of the form:

- I1
- I2
- I3
- I4
- I5
- ...

execution in a pipeline
would appear thus ==>



If each stage takes 1 clock cycle,
then throughput has increased:

- from 1 instruction per 5 cycles (sequential)
- to 1 instruction per 1 cycle (pipelined)

Q: What about Turn-around-Time aka response time?

Executing an Instruction

- Different stages of Instruction Execution:

Fetch Instruction	<i>Move the next instruction (tracked by PC) to a register</i>
Decode Instruction	<i>Identify Operator and (data) addresses</i>
Load (data)	<i>Move data into register (if needed)</i>
Execute	<i>Perform the operation</i>
Store (result)	<i>Move the resulting data to memory</i>

If separate circuitry is designed for each stage-
so that the stage take the same amount of time -
then a sequence of instructions can be executed in a pipeline

SIMD – Single Instruction, Multiple Data

Executes the **same instruction** across multiple data elements

Key Features:

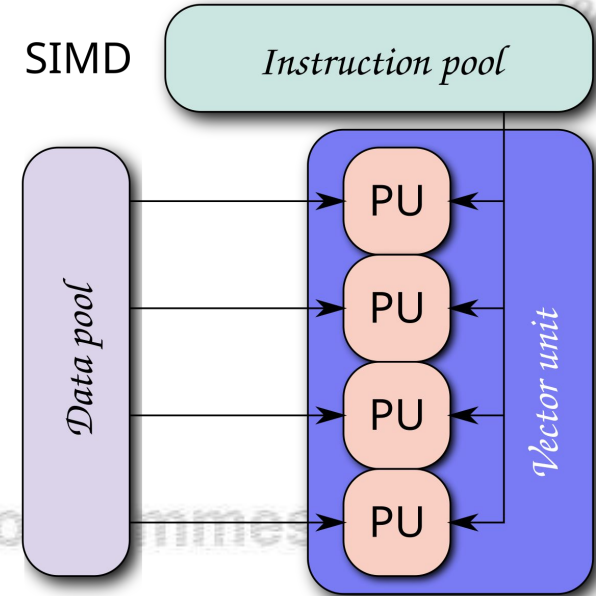
- Suitable for **data-parallel** applications
- Lockstep execution – all processing elements perform same operation simultaneously

Examples of Use:

- Vector and matrix operations
- Image filters, audio processing
- Scientific computation

Hardware Support:

- Intel AVX, ARM NEON, NVIDIA CUDA (warps/wavefronts)



MIMD – Multiple Instruction, Multiple Data

Each processor or core can execute **different instructions** on **different data**

Key Features:

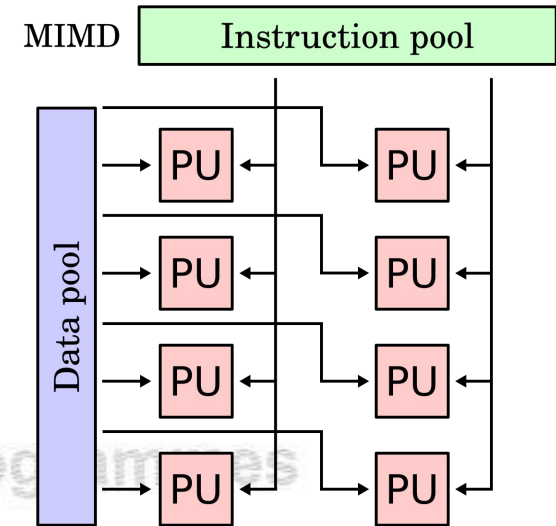
- Flexible and versatile execution model
- Ideal for task-parallel workloads

Examples of Use:

- Operating systems
- Web servers
- Multithreaded applications

Hardware Support:

- Multicore CPUs (Intel, AMD)
- Some heterogeneous compute platforms (e.g., ARM big.LITTLE)



SIMD vs. MIMD – Key Differences

Aspect	SIMD	MIMD
Instruction Stream	Single	Multiple
Data Stream	Multiple	Multiple
Synchronization	High (lockstep)	Low (independent threads)
Control Flow	Less flexible	Highly flexible
Hardware Examples	GPUs, Vector Units	Multicore CPUs, Clusters
Use Cases	Matrix ops, signal processing	Web servers, OS-level tasks
Efficiency	High for regular data structures	High for diverse workloads

Performance Model

If a program has:

- Parallel fraction = P
- Number of cores = N

Then theoretical speedup:

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

- Barriers $(1-P) \rightarrow$ Sequential part (cannot be parallelized)
- $P / N \rightarrow$ Parallel part divided across N cores

$$T_{parallel} = T_{compute} + T_{synchronization} + T_{memory\ contention}$$

$T_{compute}$: Actual useful computation time.

$T_{synchronization}$: Time spent:

- Waiting for other threads
- Barriers
- Locks / coordination

T_{memory} : Time lost due to:

- Shared memory access conflicts
- Cache coherence traffic
- Limited memory bandwidth

Memory Hierarchy Impact

CPU performance depends heavily on **cache behavior**.

Access times:

$$T_{L1} < T_{L2} < T_{L3} < T_{RAM}$$

Since RAM access is much slower, cache performance dominates CPU speed

Effective memory access time (EMAT):

$$EMAT = H \cdot T_{cache} + (1 - H) \cdot T_{memory}$$

Where:

- H = cache hit rate
- T_{cache} = cache access time
- T_{memory} = RAM access time

Higher $H \rightarrow$ lower average memory delay $\rightarrow$ better CPU performance.

Multi-core CPUs

Multi-core CPUs are effective for:

- Task-level parallelism
- Moderate data parallelism
- Branch-heavy computations
- General-purpose workloads

Multi-core performance is determined by:

$$\text{Performance} = \text{Parallel Speedup} - \text{Synchronization} + \text{Memory Overhead}$$

Even with many cores, scalability depends on:

- Parallel fraction P
- Synchronization cost
- Memory behavior
- Cache efficiency

GPGPUs

A GPGPU is designed for **massive parallel throughput computing**.

It contains:

- Thousands of lightweight cores
- Organized into **Streaming Multiprocessors (SMs)**
- Each SM executes many threads concurrently

CPU	GPU
Few powerful cores	Thousands of simpler cores
Optimized for low latency	Optimized for high throughput
Complex control logic	Minimal control logic
Large cache	Smaller cache, higher bandwidth

Execution Model – SIMT

SIMT - Single Instruction, Multiple Threads

One instruction is executed by many threads simultaneously.

Example:

$$C[i] = A[i] + B[i] \quad C[i]$$

If:

- n elements
- T threads

Each thread computes one index independently. Since operations are independent, they execute in parallel.

This model is ideal when:

- Same operation
- Different data elements

Memory Hierarchy in Multicore & GPGPU

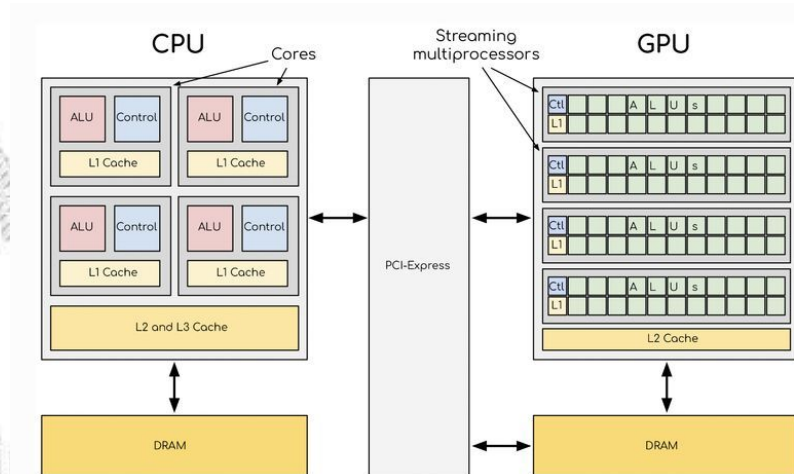
Purpose: Bridge speed gap between fast processors and slow memory

Hierarchy Levels:

- **Registers:** Fastest, smallest, processor-specific
- **L1, L2, L3 Cache:** On-chip, close to core, progressively larger and slower
- **Main Memory (RAM):** Shared DRAM, slower than cache
- **Disk/SSD:** Persistent storage, very high latency

GPU-Specific:

- **Shared Memory:** Fast, limited, user-controlled
- **Global Memory:** Accessible by all threads, high latency
- **Constant/Texture Memory:** Specialized, cached memory for specific use cases



GPU Memory Hierarchy

GPU performance strongly depends on memory behavior.

Memory Types:

- **Registers (Fastest):** Private to each thread.
- **Shared Memory (Fast):**
 - Shared among threads within a block.
 - Low latency, programmer-controlled.
- **Global Memory (Slow):**
 - Accessible by all threads.
 - High latency (hundreds of cycles).

GPU Memory Hierarchy

Performance Depends On:

Memory Coalescing

Adjacent threads should access adjacent memory locations:

Coalesced Access $\Rightarrow$ Fewer memory transactions

Minimizing Global Memory Access

Use shared memory to reduce global access.

Maximizing Occupancy

Occupancy = Active threads / Maximum supported threads

Higher occupancy hides memory latency.

GPU Memory Hierarchy

GPU performance is dominated by:

Performance = Massive Thread Parallelism + High Memory Bandwidth

Unlike CPUs (latency-optimized), GPUs are:

Throughput-Optimized Processors

They perform best when:

- Large data size n
- High parallel fraction P
- Minimal branching
- Memory access is well-structured

Memory Bandwidth Considerations

Memory Bandwidth: Maximum rate at which data can be transferred between memory and processors

Why Important?

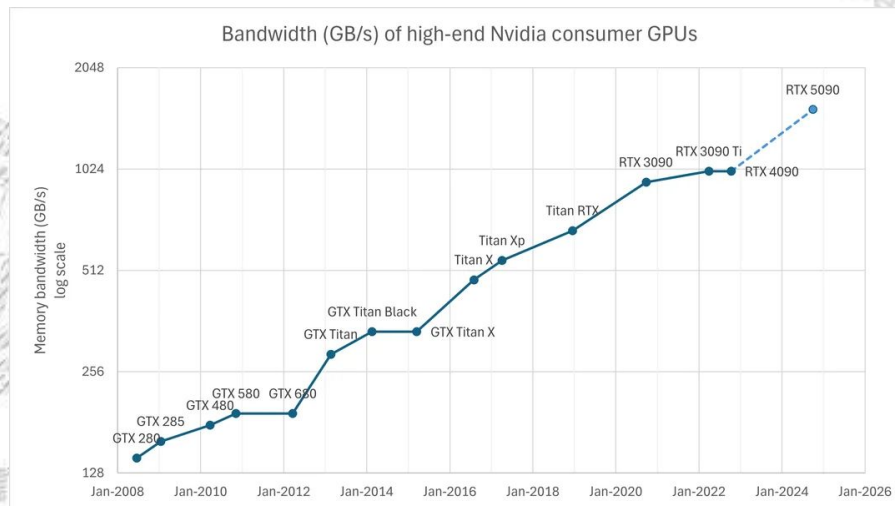
High computation speeds are meaningless without fast data access

Multicore CPUs:

- Shared bandwidth among cores → contention possible
- Cache reduces memory access latency

GPGPUs:

- Very high theoretical memory bandwidth
- Sensitive to **access patterns**
- **Coalesced access** (threads accessing adjacent memory) is critical for performance



Optimizing Memory Performance

For Multicore CPUs:

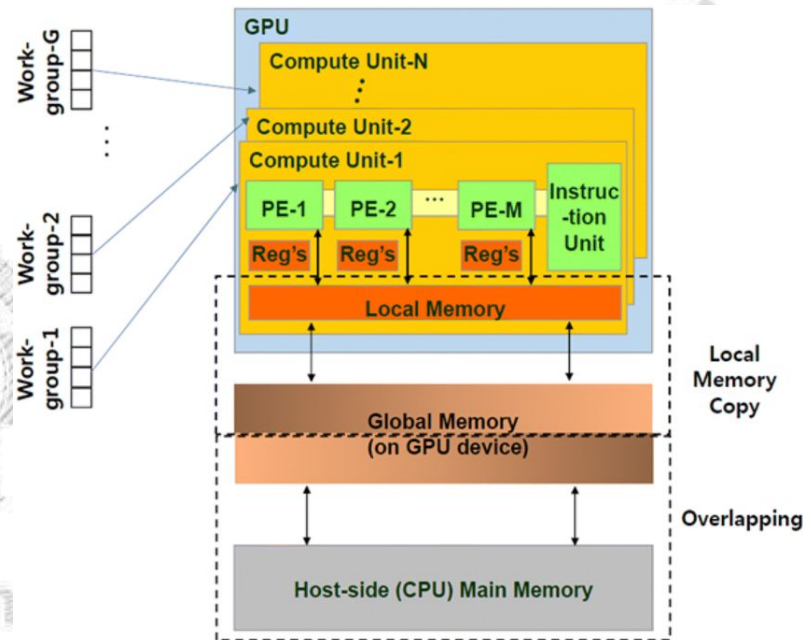
- Use **locality of reference** to benefit from caches
- Avoid **false sharing** – separate data written by different threads
- Use cache-aware data structures

For GPGPU:

- Align memory access for coalescing
- Utilize **shared memory** to avoid global memory latency
- Minimize **bank conflicts** in shared memory

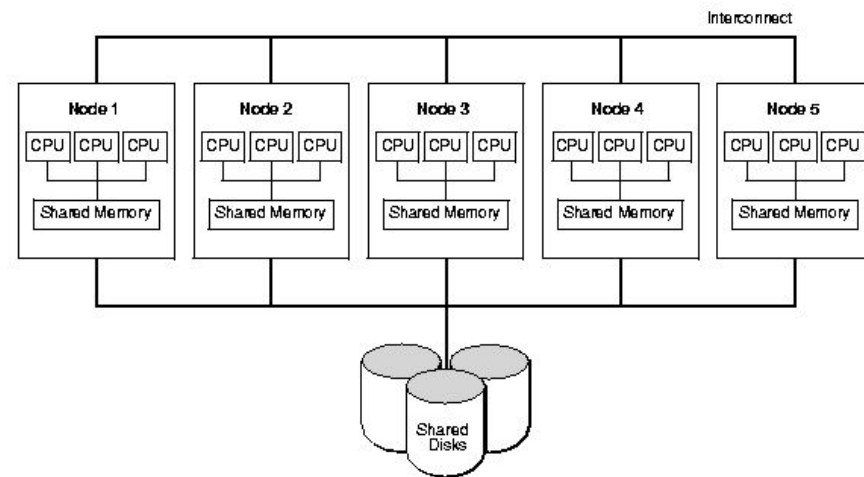
General Strategies:

- Balance **compute vs memory** loads
- Use **pipelining and tiling** techniques



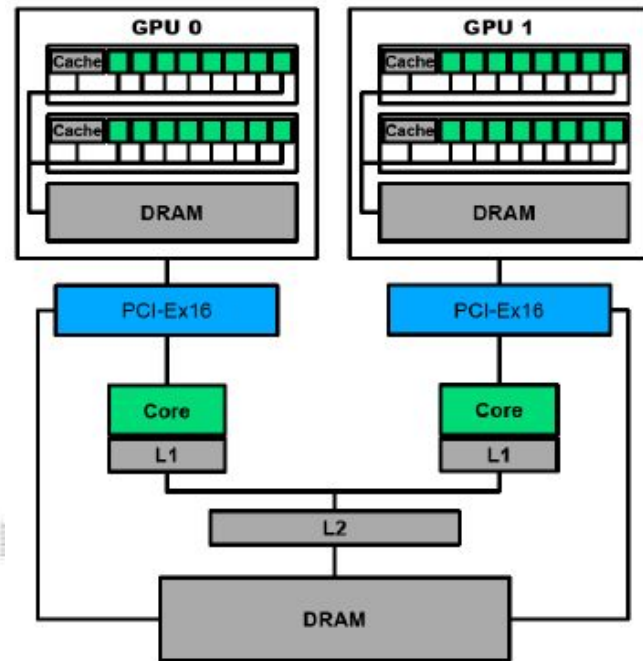
CPU-only Clusters

- Comprised solely of **multicore CPU-based nodes**.
- Suitable for:
 - Applications with control-heavy logic
 - Large in-memory databases
 - Distributed file systems and ETL pipelines
- **Benefits:**
 - More memory per node (better for large datasets)
 - Well-supported by traditional distributed frameworks (e.g., Hadoop, Spark, MPI)
 - Easier to manage and deploy
- **Limitations:**
 - Lower computational throughput
 - Slower performance in compute-intensive tasks like deep learning



GPU-Accelerated Clusters

- Consist of nodes with both CPUs and **one or more GPUs** (NVIDIA, AMD).
- Ideal for:
 - Training large-scale deep learning models (CNNs, Transformers)
 - Image/video processing, simulations
- **Benefits:**
 - Massive parallelism using thousands of CUDA cores
 - Faster matrix operations (e.g., GEMM, convolutions)
 - High floating-point throughput (TFLOPs range)
- **Limitations:**
 - Lower memory per GPU
 - Requires GPU-compatible libraries (e.g., TensorFlow GPU, CUDA, NCCL)
 - Higher hardware and maintenance cost



CPU-only vs GPU-Accelerated Clusters

Feature	CPU-only Clusters	GPU-Accelerated Clusters
Compute Units	Multicore CPUs	CPUs + Dedicated GPUs
Memory per Node	High	Lower (per GPU)
Programming Model	Threading, MPI	CUDA, OpenCL, TensorFlow, PyTorch
Power Consumption	Lower	Higher
Ideal Use Cases	Control-heavy tasks, I/O-bound jobs	Deep learning, simulations, HPC
Cost	Lower	Higher (GPUs, cooling systems, etc.)

GPU/CPU Clusters

A **cluster** is a collection of interconnected computing nodes.

Each node contains:

- One or more **multi-core CPUs**
- Possibly one or more **GPUs**
- Its own **local memory (RAM)**

Nodes are connected using a **high-speed network** (e.g., InfiniBand, Ethernet).

Distributed Memory Model

Unlike shared-memory systems:

$$M_1 \neq M_2 \neq \dots \neq M_N$$

Each node has its **own private memory**.

To share data:

$$\text{Node}_i \rightarrow \text{Node}_j$$

Data must be explicitly communicated over the network.

GPU/CPU Clusters

Communication Model

Total execution time:

$$T_{total} = T_{computation} + T_{communication}$$

α - β Communication Model

Communication cost is modeled as:

$$T_{comm} = \alpha + \beta m$$

Where:

- α = latency (startup cost per message)
- β = transfer time per byte
- m = message size (bytes)

GPU/CPU Clusters

Scalability Models

Strong Scaling

- Problem size fixed
- Increase number of processors N

Ideal case:

$$T(N) = \frac{T(1)}{N}$$

Speedup:

$$S(N) = N$$

But in reality:

$$T(N) = \frac{T(1)}{N} + T_{comm}$$

Communication overhead prevents perfect linear scaling.

GPU/CPU Clusters

Scalability Models

Weak Scaling

- Increase processors
- Increase problem size proportionally

If problem size per processor remains constant:

$$T(N) \approx T(1)$$

Ideal weak scaling maintains constant execution time.

However:

$$T(N) = T_{comp} + T_{comm}$$

As N increases, communication may grow and degrade performance.

GPU/CPU Clusters

Performance Limitation

Cluster efficiency:

$$E(N) = \frac{S(N)}{N}$$

Where:

- $E(N)$ = parallel efficiency
- $S(N)$ = speedup using N nodes
- N = number of processors/nodes

As N increases:

- Communication increases
- Synchronization increases
- Network contention increases

Thus: $E(N) \downarrow$ as $N \uparrow$

GPU/CPU Clusters

For distributed GPU/CPU clusters, overall performance can be modeled as:

Performance = $f(\text{Compute Capacity, Memory Hierarchy, Communication Cost, Load Balancing})$

This represents the **key determinants of scalability and efficiency** in cluster-based systems.

Total compute power across all nodes:

$$C_{total} = N_{nodes} \times (CPU_{cores} + GPU_{cores})$$

Where:

- N_{nodes} = number of cluster nodes
- CPU_{cores} = processing power per node (CPU)
- GPU_{cores} = GPU compute power per node

Higher compute capacity increases raw throughput.

GPU/CPU Clusters

Memory Hierarchy

Each node has:

- Local cache
- Local RAM
- GPU memory

Memory performance impacts:

$$T_{compute} \propto EMAT$$

Effective memory access time:

$$EMAT = H \cdot T_{cache} + (1 - H) \cdot T_{memory}$$

Where:

- EM AT = Effective Memory Access Time
- H = Cache hit rate
- T_{cache} = Cache access time
- T_{memory} = Main memory (RAM) access time

Poor locality increases computation time.

GPU/CPU Clusters

Communication Cost

Since clusters use **distributed memory**, communication cost is critical.

$$T_{comm} = \alpha + \beta m$$

Where:

- α = latency
- β = per-byte transfer cost
- m = message size



Total execution time:

$$T_{total} = T_{compute} + T_{comm}$$

If communication dominates:

Scalability ↓

GPU/CPU Clusters

Load Balancing

Let:

W_i = work assigned to node

Parallel execution time:

$$T_{parallel} = \max(W_1, W_2, \dots, W_N)$$

If workload is uneven:

$$T_{parallel} \uparrow \Rightarrow \text{Efficiency} \downarrow$$

Combined Cluster Performance Model

$$T_{total} = \frac{T_{compute}}{N} + T_{comm} + T_{imbalance}$$

Where:

$T_{imbalance}$ = delay due to uneven work distribution

GPU/CPU Clusters

Performance improves when:

- Compute capacity $\uparrow$
- Memory latency $\downarrow$
- Communication cost $\downarrow$
- Workload evenly distributed



Cluster performance is limited not just by computation, but by **communication and imbalance overhead**, especially at large scale.

Resource Management

Modern GPU/CPU clusters must efficiently manage:

- CPU cores → **(Multi-core CPUs)**
- GPU threads → **(GPUs / Accelerators)**
- Memory → **(All systems: CPU, GPU, Clusters)**
- Network bandwidth → **(Clusters / Distributed systems)**
- Storage I/O → **(Data centers / HPC systems)**

Poor resource management directly reduces scalability and efficiency.

Resource Management - Load Balancing

- Multi-core CPUs
- GPUs
- CPU/GPU Clusters
- Distributed systems

Goal:


$$\text{Minimize } \max(T_1, T_2, \dots, T_N)$$

Where:

- T_i = execution time on processor i

If one processor takes longer → overall system waits → efficiency drops.

Resource Management - Scheduling

- Operating Systems (CPU scheduling)
- GPU thread schedulers
- Cluster job schedulers


$$T_{\text{completion}} = \max(\text{Finish time of all tasks})$$

Schedulers aim to:

- Reduce idle processors
- Reduce context switching
- Reduce communication delay (clusters)

Resource Management - Memory

- ❖ Multi-core CPUs → Cache coherence issues
- ❖ GPUs → Shared/global memory optimization
- ❖ Clusters → Distributed memory management

Minimize:

- Memory access latency

Maximize:

- Cache hit rate

Avoid:

- Memory contention
- False sharing (CPUs)
- Bandwidth saturation (GPUs & Clusters)



Summary

Feature	Multi-core CPU	GPU	Cluster
Memory Model	Shared	Shared (device memory)	Distributed
Best For	Task parallelism	Data parallelism	Massive distributed workloads
Core Count	Few (4–64)	Thousands	Thousands to millions
Bottleneck	Cache & synchronization	Memory bandwidth	Network communication
Latency	Low	Higher per thread	High (network)
Throughput	Moderate	Very High	Extremely High



M2: Distributed ML



Parallel ML Algorithms



1. *CNN*
2. *Gradient Descent/SGD*
3. *SVM*
4. *k-Means*
5. *kNN,*
6. *Decision Trees/Random Forests*

Convolutional Neural Networks (CNN)

A **Convolutional Neural Network (CNN)** is a deep learning architecture designed to process **grid-structured data**, especially images.

An image can be represented as:

$$X \in \mathbb{R}^{H \times W \times C}$$

Where:

- H - Height
- W - Width
- C - Number of channels (e.g., 3 for RGB)

CNNs extract **spatial features** using convolution operations instead of fully connected layers.

Convolutional Neural Networks (CNN)

Instead of connecting every neuron to all pixels, CNNs use:

Local Receptive Fields: Each neuron sees only a small region (e.g., 3×3).

Weight Sharing: Same kernel slides across the image.

$$Y_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X_{i+m,j+n} \cdot W_{m,n}$$

Where:

- X = Input
- W = Kernel
- k = Kernel size
- $Y_{i,j}$ = Output feature

*This produces **feature maps** detecting edges, textures, patterns, etc.*

Structure of CNN

A CNN typically contains:

1. **Convolution Layers:** Applies learnable filters to extract important features (edges, textures, patterns) from the input image.
2. **Activation (ReLU):** Introduces non-linearity by converting negative values to zero, helping the network learn complex patterns.
3. **Pooling:** Reduces the spatial size of feature maps to decrease computation and control overfitting.
4. **Fully Connected Layers:** Combines extracted features to perform final classification or prediction.

Multiple convolution layers extract hierarchical features:

- Early layers → edges
- Middle layers → shapes
- Deep layers → objects

Why Parallelism is Required in CNN

Massive Computational Cost

From complexity:

$$\text{Complexity} = O(HWCK^2F)$$

Where:

- H,W = spatial size
- C = input channels
- k = kernel size
- F = number of filters

For large models:

- Millions to billions of parameters
- Billions of floating-point operations (FLOPs)

Sequential CPU execution becomes extremely slow.

Why Parallelism is Required in CNN

Independence in Computation

Each output pixel:

$Y_{i,j}$ is independent of $Y_{p,q}$

This means:

- Different pixels can be computed simultaneously.
- Different filters can run in parallel.
- Different images in a batch can run in parallel.

*CNN structure naturally exposes **massive parallelism**.*

Why Parallelism is Required in CNN

Training Requires Repeated Computation

Training involves:

- Forward pass
- Backward pass
- Gradient computation
- Parameter updates



For large datasets (e.g., millions of images):

Training time without parallelism → weeks or months.

Parallel hardware reduces this to hours or days.

Why Parallelism is Required in CNN

Memory Bandwidth Requirement

CNN operations involve:

- Large tensor multiplications
- Frequent memory access



Parallel systems (GPUs) provide:

- High memory bandwidth
- Thousands of threads

Which matches CNN workload.

Why Parallelism is Required in CNN

CNN convolution = structured matrix multiplication

Matrix multiplication is:

- Highly parallelizable
- Optimized on GPUs
- Efficient on distributed clusters



Thus:

CNN efficiency $\propto$ Parallel Compute Resources

Single-channel convolution

$$Y_{i,j} = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} X_{i+m,j+n} \cdot W_{m,n}$$

Where:

- $X \rightarrow$ Input feature map of size $H \times W$
- $W \rightarrow$ Kernel (filter) of size $k \times k$
- $Y_{i,j} \rightarrow$ Output at spatial position (i,j)
- $m,n \rightarrow$ Kernel indices
- $k \rightarrow$ Kernel size

At every output position:

- A **local receptive field** of size $k \times k$ is extracted.
- Each element is multiplied by corresponding kernel weight.
- Results are summed $\rightarrow$ dot product.

*So convolution = **sliding dot product**.*

Multi-Channel Convolution

Multi-Channel Convolution — Where Channel Summation Appears

Where:

$$Y = \sum_{c=1}^C X_c * W_c$$

- $C \rightarrow$ Number of input channels
- $X_c \rightarrow$ Channel c of input
- $W_c \rightarrow$ Corresponding kernel for channel c
- $*$ $\rightarrow$ Convolution operation

Each channel produces a partial output.

All channels are summed $\rightarrow$ final feature map.

Computational Complexity

Given:

- Input size = $H \times W$
- Kernel size = $k \times k$
- Channels = C
- Filters = F

For **one output pixel of one filter**:

$$\text{Operations} = C \times k^2$$

For **entire feature map**:

$$H \times W \times C \times k^2$$

For **F filters**:

$$O(HWCk^2F)$$

Intra-layer Parallelism

Each output pixel:

$Y_{i,j}$ is independent of $Y_{p,q}$

So:

- Thousands of GPU threads compute different pixels simultaneously.
- No dependency between spatial locations.

This matches GPU architecture.

Data Parallelism

If batch size = B

Split across G GPUs:

Each GPU processes $\frac{B}{G}$

Where:

- Every GPU has a **full copy of the model**.
- Each GPU computes gradients on its mini-batch.
- Gradients are **synchronized (averaged)** after the backward pass.
- Widely used in **distributed deep learning training**.
- Improves training speed without changing model structure.

Model Parallelism

Split filters:

If total filters (or parameters) = **F**

Split across **P** processors:

Each processor computes $\frac{F}{P}$ filters

Where:

- Model is **partitioned across devices**.
- Each processor handles part of the model.
- Activations must be communicated between processors.
- Useful when the **model is too large to fit in one GPU memory**.
- Common in very large models (e.g., large transformers).

Gradient Descent (GD)

Gradient Descent is a fundamental optimization algorithm used to train machine learning models.

Its purpose is to minimize a loss (cost) function by iteratively adjusting the model parameters in the direction that reduces error.

$$J(\theta) = \frac{1}{n} \sum_{i=1}^n L(f(x_i, \theta), y_i)$$

Where:

- $\theta \rightarrow$ model parameters (weights)
- $n \rightarrow$ number of training samples
- $x_i \rightarrow$ input sample
- $y_i \rightarrow$ true label
- $f(x_i, \theta) \rightarrow$ model prediction
- $L(\cdot) \rightarrow$ loss function
- $J(\theta) \rightarrow$ average training error

The objective is:

$$\theta^* = \arg \min_{\theta} J(\theta)$$

This means we want the parameter values that minimize the total error.

Gradient Descent (GD)

The gradient of the loss function:

$$\nabla J(\theta)$$

gives the direction of steepest increase.

To minimize the function, we move in the opposite direction:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta_t)$$

Where:

- $\eta \rightarrow$ learning rate
- $t \rightarrow$ iteration number

This iterative update continues until convergence.

Gradient Descent (GD)

- **Almost all modern ML models** are trained using **Gradient Descent** or its variants (SGD, Adam, RMSProp).
- **Deep learning** is entirely based on **gradient-based optimization** to update millions/billions of parameters.

Why Computation Becomes Extremely Large

1. **Large Datasets (N samples)**
 - Gradient must be computed over massive data → high data processing cost.
2. **High-Dimensional Parameter Space (d parameters)**
 - Each update requires computing partial derivatives for all parameters → $\nabla L \in \mathbb{R}^d$
3. **Deep Neural Networks**
 - Multiple layers → forward + backward propagation
 - Complexity increases with depth and width

Why GD is Computationally Heavy

Gradient of loss:

$$\nabla J(\theta) = \frac{1}{n} \sum_{i=1}^n \nabla L_i$$

Computation requires:

1. **Forward pass** for each sample $x_i \rightarrow$ compute predictions
2. **Backward pass** for each sample $\rightarrow$ compute ∇L_i
3. **Summation** over all sample gradients

When Cost Becomes Extremely High

- **Large dataset:** $n \sim$ millions of samples
- **High-dimensional model:** $\theta \sim$ millions of parameters

$$\text{Total cost} \propto n \times \text{model size}$$

Parallel Computation Model for Gradient Descent

- In **batch gradient descent**, the gradient for each sample is **independent**:

$$\nabla L_1, \nabla L_2, \dots, \nabla L_n$$

- No sample depends on another for its gradient calculation.

Parallelization Strategy

1. **Distribute the dataset** across multiple processors or GPUs.
 - Example: If batch size $B=1024$ and 4 processors $\rightarrow$ each processes 256 samples.
2. **Compute local gradients** for each subset independently.
3. **Aggregate gradients** after computation using a **parallel reduction**:

Parallel Computation Model for Gradient Descent

Assume

- **P processors**
- Dataset split into **P subsets**: $D_1, D_2, \dots, D_P$
- Each processor computes gradients for its subset:

$$\nabla J(\theta) = \frac{1}{n} \sum_{p=1}^P \sum_{i \in D_p} \nabla L_i$$

How It Works

1. Each processor performs:
 - **Forward pass** for samples in D_p
 - **Backward pass** for samples in D_p
2. Compute **local gradient sum** for its subset
3. **Parallel reduction**: sum local gradients across all processors

Parallel Computation Model for Gradient Descent

Parallel Time Approximation

$$T_{parallel} = \frac{T_{compute}}{P} + T_{communication}$$

Where:

- $T_{compute}$ → time to compute gradients
- P → number of processors
- $T_{communication}$ → time to aggregate gradients

Increasing **P** reduces computation time per processor.

Communication overhead ($T_{communication}$) may dominate if too many processors are used.

Optimal speedup balances **computation vs communication**.

Stochastic Gradient Descent (SGD)

Instead of computing gradients over the **entire dataset**, use a **mini-batch B**.

Update rule:

$$\theta_{t+1} = \theta_t - \eta \frac{1}{|B|} \sum_{i \in B} \nabla L_i$$

Where:

- $B \rightarrow$ mini-batch of samples
- $|B| \rightarrow$ size of mini-batch
- $\eta \rightarrow$ learning rate

Advantages

- Reduces **computation per iteration**
- Enables **faster updates** and **more frequent learning**
- Works well with **large datasets**

By using only a small subset of data per update, SGD trades exact gradient accuracy for faster, more scalable training.

Parallel Stochastic Gradient Descent (SGD)

Communication in Distributed Training

Total Time in a Cluster

$$T_{total} = T_{compute} + T_{communication}$$

Communication Model (α - β Model)

$$T_{comm} = \alpha + \beta m$$

Where:

- $\alpha \rightarrow$ **latency cost** (fixed overhead per message)
- $\beta \rightarrow$ **transfer time per byte**
- $m \rightarrow$ **gradient size** (number of parameters)

Large models $\rightarrow$ large $m \rightarrow$ **communication dominates training time**

Optimization of communication (e.g., gradient compression, All-Reduce) is crucial for **scalable distributed training**

Support Vector Machine (SVM)

Support Vector Machine (SVM) is a supervised learning algorithm used for classification and regression.

Goal: Find a **hyperplane** that separates two classes with **maximum margin** while allowing for some misclassification (ξ_i).

Mathematically, the **optimization problem** is:

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i$$

Subject to constraints: $y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0$

Where:

- w = weight vector (hyperplane normal)
- b = bias term
- ξ_i = slack variables (allow margin violations)
- C = regularization parameter (penalty for misclassification)
- x_i, y_i = training samples and labels

Computational Complexity – SVM

Kernel Matrix Computation

- **Kernel:** $K(x_i, x_j)$ for all pairs of samples
- **Kernel matrix size:** $K \in \mathbb{R}^{n \times n}$
- **Time Complexity:** $O(n^2)$



*Computing the kernel matrix dominates cost for large datasets; **parallelization across kernel entries** significantly reduces computation time.*

Parallelization in SVM

Kernel Computation

- Kernel values between pairs (x_i, x_j) are independent:

$$K(x_i, x_j) \quad \forall i, j$$

- Can compute **blocks of the kernel matrix in parallel** across multiple cores or GPUs.
- This reduces the time from $O(n^2)$ sequentially to roughly $O(n^2/P)$ with P processors.

Training Parallelism

- Training methods like **SMO (Sequential Minimal Optimization)** update support vectors iteratively, but parts of computation (gradient updates, kernel block computations) can run concurrently.
- **Mini-batch approaches** or distributed SVMs allow larger parallelization.

Kernel computation is the most parallelizable part, while training updates have limited but still useful parallel opportunities in large-scale or distributed setups.

Why Parallelism is Needed in SVM

Challenge

- Large datasets: $n \sim 10^5 - 10^6$
- Kernel matrix size: $n \times n \rightarrow$ **huge memory and computation requirement**
- Sequential computation: **slow and memory-intensive**

Benefits of Parallelism

1. **Faster kernel matrix evaluation**
 - Compute blocks independently on multiple processors/GPUs
2. **Efficient gradient updates**
 - Parallel updates of support vector contributions
3. **Scalable SVM training**
 - Enables training on **clusters** or **GPU farms** for large-scale datasets

Parallelism is essential to make SVM training feasible and efficient on modern, large datasets.

SVM Parallelization

Component	Complexity	Parallelization Potential
Kernel matrix computation	$O(n^2)$	High (compute independent blocks)
Training (SMO/QP)	Depends on iterations	Partial (gradient & kernel computations)

SVM training benefits from **data-level parallelism** (kernel computation) and **task-level parallelism** (optimization steps), especially on large datasets or distributed systems.

k-Means

k-Means is an unsupervised clustering algorithm that partitions n data points into k clusters by minimizing the **within-cluster sum of squares (WCSS)**:

$$J = \sum_{i=1}^n \min_k \|x_i - \mu_k\|^2$$

Where:

- $x_i \rightarrow$ data point i
- $\mu_k \rightarrow$ centroid of cluster k
- $\|x_i - \mu_k\|^2 \rightarrow$ squared Euclidean distance

k-Means

Algorithm Steps (Iterative Process)

Assignment Step:

Assign each data point x_i to the **nearest centroid** μ_k .

Update Step:

Recompute centroids:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

Where:

- $C_k \rightarrow$ set of points assigned to cluster k
- $|C_k| \rightarrow$ number of points in cluster k

Computational Complexity of k-Means

Assignment Step

$$O(nkd)$$

- For each of the n data points
- Compute distance to each of the k centroids
- Each distance computation costs $O(d)$ in d -dimensional space

Update Step

$$O(nd)$$

- Sum all points belonging to each cluster
- Recompute centroids by averaging
- Total cost proportional to number of points and dimensions

Overall Complexity

$$O(nkd \times \text{iterations})$$

- Algorithm repeats until convergence
- Runtime depends on number of iterations required

Parallelization in k-Means

Step 1 – Assignment (Data Parallelism)

Distance computations:

$$\|x_i - \mu_k\|^2 \quad \forall i, k$$

can be performed **concurrently**.

Where:

- $x_i \rightarrow$ data point i
- $\mu_k \rightarrow$ centroid of cluster k
- $i=1,2,\dots,n$
- $k=1,2,\dots,K$
- $\|x_i - \mu_k\|^2 \rightarrow$ squared Euclidean distance

Since each pair (i,k) is independent, these computations can be executed **in parallel across cores or GPU threads**.

Parallelization in k-Means

Step 2 – Centroid Update (Parallel Reduction)

For cluster k:

$$\mu_k = \frac{1}{|C_k|} \sum_{x_i \in C_k} x_i$$

Where:

- $\mu_k \rightarrow$ centroid of cluster k
- $C_k \rightarrow$ set of data points assigned to cluster k
- $|C_k| \rightarrow$ number of points in cluster k
- $x_i \rightarrow$ data point belonging to cluster k

Parallel Strategy

1. Split cluster points among threads
2. Each thread computes a **local sum**
3. Combine partial sums
4. Divide by $|C_k|$ to get updated centroid

Centroid recomputation is efficiently parallelizable using reduction operations on multi-core CPUs and GPUs.

Why Parallelization Helps in k-Means

Challenges

- Large datasets ($n \gg 10^5$) → Sequential computation becomes slow
- High-dimensional data ($d \gg 10$) → Each distance computation is expensive

How Parallelization Improves Performance

- **Assignment Step** → Exploits **data-level parallelism**
 - Distances for multiple points computed simultaneously
- **Update Step** → Uses **reduction-level parallelism**
 - Parallel summation of cluster points
- **Distributed Setting (Clusters)**
 - Data batches distributed across nodes
 - Local centroid updates combined globally

k-Nearest Neighbors (kNN)

k-Nearest Neighbors (kNN) is a non-parametric, instance-based learning algorithm used for classification and regression.

Unlike models such as SVM or neural networks:

- **No explicit training phase**
- The entire training dataset is stored
- Prediction is done by comparing a test sample to all training samples

It is called a **lazy learner** because computation is deferred until prediction time.

k-Nearest Neighbors (kNN)

For a test sample $x \in \mathbb{R}^d$, compute its distance to every training sample x_i :

$$d(x, x_i) = \sum_{j=1}^d (x_j - x_{ij})^2$$

Where:

- $x \in \mathbb{R}^d \rightarrow$ test sample in ddd-dimensional space
- $x_i \rightarrow$ i-th training sample
- $x_j \rightarrow$ j-th feature of test sample
- $x_{ij} \rightarrow$ j-th feature of training sample i
- $d(x, x_i) \rightarrow$ squared Euclidean distance

Prediction Steps

1. Compute distances to all training samples
2. Sort distances
3. Select the k smallest

Final Prediction:

- **Classification** $\rightarrow$ Majority vote among nearest neighbors
- **Regression** $\rightarrow$ Mean value of nearest neighbors

Computational Complexity of k-NN

Training Phase

$O(1)$

- No model training or parameter estimation
- Only storing the dataset

If Full Sorting is Performed:

$O(nd + n \log n)$

If Partial Selection is Used (Heap / Quickselect):

$O(nd)$

Prediction Phase

For each test sample:

- Compute distance to all n training samples
- Each distance computation costs $O(d)$

k-NN has negligible training cost but expensive prediction cost, especially for large n and high-dimensional data.

Total Distance Cost:

$O(nd)$

Where:

- $n \rightarrow$ number of training samples
- $d \rightarrow$ number of features

Why k-NN Becomes Expensive

Example Scenario

If:

$$n = 10^6, d = 100$$

Then for **one test sample**, distance computations required:

$$10^6 \times 100 = 10^8 \text{ operations}$$

Why This is Heavy

- 100 million arithmetic operations per prediction
- Must repeat for every test sample
- Becomes impractical for real-time systems

k-NN scales poorly with large datasets, making parallelization or approximate methods necessary for efficiency.

Parallelization in k-NN

Each distance:

$$d(x, x_i)$$

is independent of others → **k-NN is embarrassingly parallel.**

Where:

- $x \rightarrow$ test sample
- $x_i \rightarrow$ i-th training sample
- $d(x, x_i) \rightarrow$ distance between test and training sample

Parallelization in k-NN

Parallel Strategies

Data-Level Parallelism

If P processors are used:

$$T_{\text{parallel}} \approx \frac{nd}{P}$$

Where:

- $n \rightarrow$ number of training samples
- $d \rightarrow$ feature dimension
- $P \rightarrow$ number of processors
- $T_{\text{parallel}} \rightarrow$ parallel prediction time

Each processor computes distances for n / P training samples

Parallelization in k-NN

Parallel Strategies

Parallel Reduction for k Selection

After computing all distances $d(x, x_i)$, we must select the **k smallest distances**.

Why This Matters

- Distance computation is parallel
- But selecting the **top-k nearest neighbors** can become a bottleneck
- Efficient parallel selection is required for scalability.

k-Nearest Neighbors (kNN)

Component	Best Hardware	Reason
Distance computation	GPU	Massive independent arithmetic
Small datasets	Multi-core CPU	Lower overhead
Very large datasets	Cluster	Distribute training data

Decision Tree

A **Decision Tree** is a supervised learning algorithm used for **classification and regression**.

It recursively partitions the dataset into subsets based on feature values, aiming to maximize **class purity** at each split.

Choose the feature that provides the **best separation of data** at each node.

How It Works

- Start with the entire dataset at the root node
- Select the best feature to split the data
- Create child nodes based on split condition
- Repeat recursively until stopping criteria is met

Decision Tree - Splitting Criterion

Entropy

For a dataset S with c classes:

$$H(S) = - \sum_{i=1}^c p_i \log p_i$$

Where:

- p_i = probability of class i in dataset S
- Entropy measures **impurity**
- $H(S) = 0 \rightarrow$ pure node
- Maximum entropy $\rightarrow$ uniform distribution

Decision Tree - Splitting Criterion

Information Gain

After splitting dataset SSS using feature A:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

Where:

- S_v = Subset where feature $A=v$
- $|S_v|/|S|$ = Weight of subset
- $H(S_v)$ = Entropy after split

*Information Gain measures **reduction in entropy** after splitting.*

*The feature with **maximum Information Gain** is chosen for the split.*

Computational Complexity of Decision Trees

At Each Node

Operations performed:

1. Evaluate all **d features**
2. For each feature, examine possible split points
3. Compute impurity (Entropy/Gini) for each split

Time Complexity Per Node

If:

- n = number of samples at that node
- d = number of features

Then computational cost per node: $O(nd)$

- For each feature $\rightarrow$ scan all n samples
- Total work $\rightarrow d \times n$

Computational Complexity of Decision Trees

Overall Tree Complexity

If the tree depth is h :

$$\text{Total Cost} \approx O(ndh)$$

(Each level evaluates d features across the available n samples.)

For:

- **Balanced tree** $\rightarrow h \approx \log n$

$$O(nd \log n)$$

(Logarithmic depth reduces total split evaluations.)

- **Worst case (skewed tree)** $\rightarrow h \approx n$

$$O(n^2d)$$

(Unbalanced splits repeatedly process nearly the entire dataset.)

Random Forest

Random Forest is an ensemble learning method that combines multiple decision trees to improve prediction accuracy and reduce overfitting.

For input x :

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T f_t(x)$$

Where:

- T = number of decision trees in the forest
- $f_t(x)$ = prediction of the t -th tree
- $\hat{y}$ = aggregated output (average for regression, majority vote for classification)

Random Forest uses:

- **Bootstrap sampling** for each tree (different subset of data)
- **Random feature selection** at splits to increase diversity

Why Random Forests Are Naturally Parallel

Random Forests are ensembles of multiple decision trees trained on different subsets of data and features. Their structure makes parallelization very efficient:

Independence During Training

Each tree $f_1, f_2, \dots, f_T$ is trained **independently** using:

- **Bootstrap sampling:** a random subset of the training data per tree.
- **Random feature selection:** only a subset of features is considered at each split.

Implication:

- There are **no dependencies between trees**, so each tree can be trained on a separate processor or core without waiting for others.
- This allows **linear scaling** with the number of available processors, up to TTT.

Why Random Forests Are Naturally Parallel

Independence During Prediction

After training, each tree produces a prediction $f_t(x)$ for input x .

- Predictions from all trees can be computed **simultaneously**.
- Final output is obtained via:
 - **Majority vote** for classification
 - **Average** for regression

Implication:

- Tree predictions are independent until aggregation, making the prediction phase **embarrassingly parallel**.
- This is especially useful for real-time or large-scale inference.

Parallel Training Model

Assumptions:

- Serial training time per tree: T_{serial}
- Number of processors: P

Parallel training time:

$$T_{\text{parallel}} \approx \frac{T_{\text{serial}}}{P}$$

- Each processor trains **one or more trees independently**.
- No synchronization is needed between trees until aggregation of results.

Ideal Speedup:

$$S_{\text{max}} \approx P (\text{if number of trees } T \geq P)$$

- Linear scaling is possible when the number of trees is at least the number of processors.
- Perfect for **multi-core CPUs, GPUs, or distributed clusters**.

Parallel Prediction in Random Forests

- Assign each tree (or group of trees) to a processor p , $p=1,2,\dots,P$

$f_t(x)$ computed independently for all $t \in$ trees on processor p

- Aggregation step after all processors finish:

$$\hat{y} = \frac{1}{T} \sum_{p=1}^P \sum_{t \in T_p} f_t(x)$$

Where T_p = set of trees assigned to processor p , $\bigcup_{p=1}^P T_p = \{1, 2, \dots, T\}$

Speedup:

- Ideal parallel time for prediction:

$$T_{\text{parallel}} \approx \frac{T_{\text{serial}}}{P}, \quad S_{\text{max}} \approx P \quad (\text{if } T \geq P)$$

- Only synchronization: aggregation (voting or averaging).

Comparison of Parallelization Suitability

Algorithm	Data Parallelism	Task Parallelism	Cluster-Friendly	GPU-Friendly
CNN	Excellent	Moderate	Excellent	Excellent
GD / SGD	Excellent	Moderate	Excellent	Excellent
SVM	Moderate	Limited	Moderate	Limited
k-Means	Excellent	Limited	Good	Excellent
kNN	Excellent	Limited	Good	Excellent
Decision Trees	Moderate	Good	Good	Moderate
Random Forest	Excellent	Excellent	Excellent	Moderate



Distributed Training Strategies



1. *Data parallelism*
2. *Model parallelism*
3. *Pipeline parallelism*
4. *gradient checkpointing*
5. *Mixed precision training*

Data Parallelism (DP)

Data Parallelism is a **parallelization strategy** where the **same model** is replicated across multiple devices (CPUs, GPUs, or TPUs) to **process different subsets of the dataset simultaneously**.

Goal:

- Reduce total training time.
- Efficiently utilize hardware resources.
- Keep model parameters synchronized across devices.

Let total batch size = B

Let number of devices = N

Split batch across devices:

$$B_i = \frac{B}{N}, \quad i = 1, 2, \dots, N$$

Each device i processes its mini-batch B_i **independently**.

*Computation on each mini-batch does **not depend on other mini-batches**, so it can run concurrently → true parallelism.*

Local Gradient Computation

For a single device i , compute the gradient of the loss function:

$$\nabla J_i(\theta) = \frac{1}{B_i} \sum_{j \in B_i} \nabla L_j(\theta)$$

Where:

- $L_j(\theta)$ = loss for sample j in the mini-batch.
- θ = model parameters.
- $\nabla J_i(\theta)$ = **local gradient** on device i .

Each device independently computes the contribution of its mini-batch to the overall gradient.

*This is a **data-parallel operation**, since mini-batches are disjoint.*

Global Gradient Aggregation

Once each device i computes its local gradient $\nabla J_i(\theta)$, the **global gradient** is computed as:

$$\nabla J(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla J_i(\theta)$$

$N \rightarrow$ number of devices

$\nabla J(\theta) \rightarrow$ gradient used for model update

How It's Done

- Performed using **All-Reduce** in GPU clusters.
- **All-Reduce**: sums the local gradients from all devices and distributes the result back to each device simultaneously.

Prevents parameter divergence: ensures all devices update their model with the same gradient.

Maintains consistency: every device keeps identical parameters θ after the update.

Parameter Update

After global gradient aggregation, model parameters are updated as:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta)$$

$\theta_t \rightarrow$ parameters at iteration t

$\eta \rightarrow$ learning rate

$\nabla J(\theta) \rightarrow$ global gradient aggregated from all devices

How It Works

- Every device applies the **same global gradient** to its local model.
- Ensures all devices **remain synchronized** after the update.

*Guarantees **correct convergence** of the training process.*

*Even though each device processes different mini-batches, **all model replicas stay identical**.*

Parallelism Benefits

Speedup

- Each device processes **$1/N$** of the total batch.
- Reduces **per-iteration computation time**.
- Ideal case: speedup $\approx N$ if **communication overhead is negligible**.

Scalability

- Adding more devices enables **larger batch sizes** or **faster training**.
- Training throughput grows **linearly with N** until **communication costs dominate**.

Hardware Utilization

- Maximizes **GPU/CPU cores** and **memory bandwidth**.
- Prevents idle cores during **forward/backward passes**.
- Efficient for large models and high-dimensional data.

Communication Overhead

$$T_{\text{comm}} = \alpha + \beta \cdot m$$

$\alpha \rightarrow$ **latency per message** (fixed cost per communication).

$\beta \rightarrow$ **time per byte** (depends on bandwidth).

$m \rightarrow$ **message size** (size of gradient vector being communicated).

- After **local gradient computation**, devices must communicate to compute the **global gradient**.
- With **too many devices**, communication time can **dominate computation**, reducing overall efficiency.
- **Optimization strategies**: gradient compression, overlap of communication and computation, hierarchical aggregation.

Data Parallelism (DP) - Example

Suppose:

- Batch size $B = 1024$
- Number of GPUs $N = 4$

Step 1 – Split mini-batches:

$$B_i = \frac{1024}{4} = 256 \quad \text{samples per GPU}$$

Step 2 – Compute local gradients:

Each GPU computes:

$$\nabla J_i(\theta) = \frac{1}{256} \sum_{j=1}^{256} \nabla L_j(\theta)$$

Step 3 – Aggregate global gradient:

$$\nabla J(\theta) = \frac{1}{4} \sum_{i=1}^4 \nabla J_i(\theta)$$

Step 4 – Update parameters:

$$\theta_{t+1} = \theta_t - \eta \nabla J(\theta)$$

*Each GPU only sees 256 samples but contributes to the **same model update** as if 1024 samples were processed sequentially.*

*Total training time is roughly **1/4 of single GPU**, minus communication overhead.*

Model Parallelism (MP)

- Model Parallelism is used when the **model is too large to fit into the memory of a single device** (GPU/CPU).
- Instead of replicating the model on multiple devices (as in DP), **the model itself is split** across devices.
- Each device holds a part of the model and performs computations only for that part.

Typical use case:

- Very deep neural networks (e.g., GPT-like transformers, ResNets with hundreds of layers).
- Extremely large embeddings (e.g., NLP models with billions of parameters).

Model Parallelism (MP)

Forward Pass

Let the model have m layers:

$$L_1, L_2, \dots, L_m$$

Split across devices:

- GPU1 $\rightarrow$ layers L_1, L_2
- GPU2 $\rightarrow$ layers L_3, L_4
- GPU3 $\rightarrow$ layers $L_5, \dots, L_m$

Input x flows sequentially:

$$y = L_m(\dots(L_2(L_1(x))))$$

- Each device computes only its assigned layers:

$$\text{Device 1: } a_2 = L_2(L_1(x)) \quad \text{Device 2: } a_4 = L_4(L_3(a_2)) \quad \text{Device 3: } y = L_m(a_4)$$

- Activations (a_i) must be **transferred** to the next device in sequence.

Model Parallelism (MP)

Backward Pass

- Gradients are computed **layer by layer in reverse order** (standard backpropagation).
- For device i :

$$\delta_i = \frac{\partial L}{\partial a_i}, \quad \theta_i^{\text{new}} = \theta_i - \eta \delta_i$$

$\delta_i \rightarrow$ gradient of loss w.r.t activations of layer i

$\theta_i^{\text{new}} \rightarrow$ updated weights for layer i

$\eta \rightarrow$ learning rate

Device Communication

- **Forward pass**: activations are sent to the next device.
- **Backward pass**: gradients are sent back to previous devices.
- Ensures **correct weight updates** across split model.

Model Parallelism (MP)

- Let total model parameters = P
- Device memory limit = M

Then **number of devices required**:

$$N_{\text{devices}} \geq \frac{P}{M}$$

- If forward computation of layer i takes t_i , total forward time:

$$T_{\text{forward}} = \sum_{i=1}^m t_i + \sum_{i=1}^{m-1} t_{\text{comm}}(a_i)$$

Where t_{comm} = communication time for passing activations between devices.

- Total training time includes **forward + backward + communication**:

$$T_{\text{total}} = T_{\text{forward}} + T_{\text{backward}} + T_{\text{communication}}$$

Comparison with Data Parallelism

Feature	Data Parallelism (DP)	Model Parallelism (MP)
What is split?	Dataset (mini-batches)	Model (layers, parameters)
Devices hold	Entire model	Part of model
Forward pass	Each device independent	Sequential across devices
Communication	Gradients aggregated (All-Reduce)	Activations and gradients passed sequentially
Best for	Small/medium models with large datasets	Very large models that don't fit on single device

Pipeline Parallelism

Extension of Model Parallelism: Model is split across multiple devices, each handling a stage of the network.

Stage-wise Division: Each device processes a subset of consecutive layers.

Micro-Batch Feeding: Input batch is divided into smaller micro-batches for sequential feeding.

Overlapping Computation & Communication: While GPU1 computes micro-batch 2 at stage 1, GPU2 can compute micro-batch 1 at stage 2 → improves throughput.

Reduces Idle Time: Devices are rarely idle compared to standard model parallelism.

Pipeline Parallelism

Scalable to Deep Models: Suitable for very deep networks that cannot fit entirely on one device.

Forward & Backward Overlap: Backpropagation for one micro-batch can start before the forward pass of the next micro-batch finishes.

Throughput Optimization: Total training time is reduced even if per-micro-batch latency is similar.

Micro-Batch Size Trade-off: Smaller micro-batches → better overlap, but higher communication overhead; larger micro-batches → more efficient computation per device.

Pipeline Parallelism

- Split the model into **stages** across multiple GPUs: $S_1, S_2, \dots, S_N$
- Forward/backward passes are executed in **pipeline fashion**:
 - While GPU1 processes batch B_2 at layer L_1 , GPU2 processes batch B_1 at layer L_2
- Overlaps computation and communication $\rightarrow$ improves throughput

Let:

- t_{S_i} = time to process stage S_i for one batch
- Pipeline efficiency factor = **efficiency** ≤ 1 (due to pipeline bubbles)

Then approximate total training time:

$$T_{\text{total}} \approx \text{pipeline efficiency} \cdot \sum_{i=1}^N t_{S_i}$$

- The sum represents **stage-wise processing time**
- Pipeline bubbles reduce efficiency at the **start and end** of training

Pipeline Parallelism

Throughput Optimization

$$T_{\text{pipeline}} \approx \max(T_1, T_2, \dots, T_n) \cdot \text{number of micro-batches}$$

$T_i \rightarrow$ time taken by **stage i** to process one micro-batch

$n \rightarrow$ total number of **stages (devices)** in the pipeline

number of micro-batches $\rightarrow$ batch size / micro-batch size

Goal:

- Balance T_i across all stages so that no stage becomes a bottleneck.
- Ensures all devices work continuously, minimizing idle time.
- Maximizes **throughput** rather than reducing per-batch latency.

If one stage is slower, it determines the pipeline speed.

Effective stage assignment and micro-batch sizing are critical for efficiency.

Gradient Checkpointing

- Backpropagation stores **activations of all layers** for computing gradients.
- Problem: **Memory grows linearly with model depth** → can exceed GPU memory for large networks.
- Solution: Store **only selected activations** (checkpoints) and recompute the rest during backward pass.
- Modern models (Transformers, CNNs, ResNets) have **hundreds of layers**.
- GPU memory is **limited**, especially for large batch sizes.
- Naive backprop requires storing all intermediate activations:

$$M_{total} \approx O(L \cdot M_{layer})$$

Where:

$L \rightarrow$ number of layers

$M_{layer} \rightarrow$ memory per layer

Gradient checkpointing reduces this significantly.

Gradient Checkpointing

Forward Pass Memory Saving (Activation Checkpointing)

- Instead of storing **all activations** for backpropagation, store only a subset called **checkpoints**.
- Activations between checkpoints are **recomputed** during the backward pass.
- Reduces memory footprint significantly, allowing **larger models or batch sizes** on limited GPU memory.

Memory Cost $\approx O(\text{number of checkpoints})$

Compute Cost $\approx O(\text{forward pass recomputation})$

- More checkpoints $\rightarrow$ higher memory usage but lower recomputation cost.
- Fewer checkpoints $\rightarrow$ lower memory usage but higher compute overhead.
- Optimal checkpoint strategy balances memory and recomputation to fit GPU memory limits.

Gradient Checkpointing

Impact on Backward Pass

- During backpropagation, intermediate activations not stored are recomputed from the nearest checkpoint.
- Ensures gradient computation remains **exact**, but adds extra computation.
- Trade-off is **memory saved vs extra forward passes**.

Combined Memory-Compute Considerations

- Memory reduction factor $\approx$ **total layers / checkpoints stored**.
- Especially useful in **very deep networks** where storing all activations is prohibitive.
- Works well with **mixed-precision training** to further reduce memory usage.

Gradient Checkpointing

Let:

- L = total number of layers
- C = number of checkpoints stored
- M_{layer} = memory required to store activations of one layer
- T_f = forward computation time per layer

Memory Usage

Without checkpointing:

$$M_{\text{full}} \approx L \cdot M_{\text{layer}}$$

With checkpointing:

$$M_{\text{checkpoint}} \approx C \cdot M_{\text{layer}}$$

Memory reduction factor:

$$\frac{M_{\text{checkpoint}}}{M_{\text{full}}} = \frac{C}{L}$$

So memory scales **linearly with number of checkpoints**, not total layers.

Gradient Checkpointing

Additional Compute Cost

Activations between checkpoints must be recomputed.

Extra computation:

$$\text{Extra Compute} \approx (L-C) \cdot T_f$$

Total effective compute time becomes:

$$T_{\text{total}} \approx T_{\text{forward}} + T_{\text{backward}} + (L-C)T_f$$

This shows:

- Fewer checkpoints $\rightarrow$ lower memory
- But $\rightarrow$ higher recomputation cost

If $C=L \rightarrow$ No recomputation, full memory usage

If $C \ll L \rightarrow$ Large memory savings, moderate compute increase

Typically, compute overhead increases modestly (~20 – 40%) while memory drops dramatically

Forward and Backward Workflow with Checkpointing

Forward Pass

- Compute outputs sequentially:

$$a_l = f_l(a_{l-1}, \theta_l)$$

- Store activations **only at checkpoint layers**
- Discard intermediate activations to save memory

If checkpoints are placed every k layers:

$$C = \frac{L}{k}$$

Only C activations remain in memory.

Backward Pass

For gradient computation:

$$\delta_l = \frac{\partial L}{\partial a_l}$$

For layers between two checkpoints:

1. Reload last checkpoint activation
2. Recompute forward activations up to layer l
3. Compute gradient:

$$\frac{\partial L}{\partial \theta_l} = \delta_l \cdot \frac{\partial a_l}{\partial \theta_l}$$

4. Move backward layer by layer

Mixed Precision Training

Mixed Precision Training uses a combination of:

- **FP16 (half precision, 16-bit)** → for forward and backward computations
- **FP32 (single precision, 32-bit)** → for maintaining stable master weights

Instead of using full 32-bit precision everywhere, it strategically reduces precision where safe.

Most deep learning operations (matrix multiplications, convolutions) are tolerant to small numerical noise.

Thus:

Memory per parameter:

FP32 → 4 bytes

FP16 → 2 bytes

Memory reduction:

$$M_{new} \approx \frac{1}{2} M_{FP32}$$

This allows:

- Larger batch sizes
- Larger models
- Reduced memory bandwidth usage

Why Mixed Precision is Needed

Modern models:

- Large parameter count
- Large batch sizes
- Deep architectures

Memory requirement:

$$M_{total} = M_{weights} + M_{activations} + M_{gradients} + M_{optimizer}$$

For very deep networks:

$$M_{total} \gg \text{GPU}$$

Mixed precision reduces:

- Activation memory
- Gradient memory
- Temporary tensor memory

Mixed Precision Training

Compute Acceleration

Modern GPUs (especially those with **Tensor Cores**) are optimized for FP16 operations.

If:

$$T_{FP32} = \text{compute time in full precision}$$

Then typically:

$$T_{FP16} < T_{FP32}$$

Speedup depends on hardware architecture.

Training Workflow

1. Forward pass → Computed in FP16
2. Loss computed in FP32
3. Backward pass → Gradients in FP16
4. Gradients converted to FP32
5. Master weights updated in FP32

Weight update:

$$\theta_{t+1}^{FP32} = \theta_t^{FP32} - \eta \nabla L^{FP32}$$

FP16 weights are refreshed from FP32 master copy.

Mixed Precision: Forward & Backward Computation Flow

Forward Pass (FP16 Compute)

$$y=f(x;\theta^{FP16})$$

- Model weights are cast from FP32 $\rightarrow$ FP16
- Activations and matrix multiplications computed in FP16
- Faster execution (Tensor Cores)
- Lower memory usage

Backward Pass (FP16 Gradients)

$$\nabla J^{FP16} = \frac{\partial L}{\partial \theta}$$

- Gradients computed in FP16
- Loss scaling may be applied to prevent underflow

Stable Weight Update (FP32 Master Copy)

Steps:

1. Convert gradients:

$$\nabla J^{FP32} = \text{Cast}(\nabla J^{FP16})$$

2. Update master weights:

$$\theta_{t+1}^{FP32} = \theta_t^{FP32} - \eta \nabla J^{FP32}$$

3. Cast updated weights back:

$$\theta_{t+1}^{FP16} = \text{Cast}(\theta_{t+1}^{FP32})$$

Mixed Precision Training

Problem: Gradient Underflow in FP16

FP16 has limited precision:

- Smallest normal positive value $\approx 2^{-14}$
- Smallest representable value $\approx 2^{-24}$

If:

$$\nabla J < 2^{-24}$$

Then it becomes:

$$\nabla J = 0$$

Small gradients vanish

No weight update

Training becomes unstable

Solution: Loss Scaling

Multiply the loss by a scale factor S:

$$L' = S \cdot L$$

Gradient Computation After Scaling

During backpropagation:

$$\nabla J' = S \cdot \nabla J$$

Gradients are now larger $\rightarrow$ safe from underflow in FP16.

Before Weight Update (Unscale Gradients)

Divide gradients by the same scale factor:

$$\nabla J = \frac{\nabla J'}{S}$$

Then update weights normally in FP32:

$$\theta_{t+1}^{FP32} = \theta_t^{FP32} - \eta \nabla J$$

Mixed Precision Training

Loss Scaling

- Scaling shifts gradients into representable FP16 range
- Division restores original magnitude
- No change to true optimization direction
- Prevents gradient underflow

Types of Loss Scaling

1. Static Scaling

- Fixed scale value S
- Simple but may overflow

2. Dynamic Scaling

- Automatically adjusts S
- Reduces overflow risk
- Used in modern deep learning frameworks